

Proč tato pravidla

Motivace za rámcem pro AI-asistovaný vývoj

Doprovodný text k dokumentu „Pravidla pro AI-asistovaný vývoj“

Krátké zamyšlení nad tím, z čeho rámec vyrostl — principy, ne návod.

1. Rychlost není totéž co zvládnutí

AI nástroje pro asistenci při vývoji dramaticky zvyšují rychlost, s jakou vzniká kód. To je jejich hlavní přínos — a zároveň jejich hlavní riziko. Rychlost bez porozumění nevede k lepšímu softwaru; vede k softwaru, který nikdo nedokáže bezpečně udržovat ani obhájit.

Jádro problému je nenápadné: AI generuje kód, který je syntakticky správný a projde základními testy — ale sám o sobě neodráží architektonická rozhodnutí, model hrozeb ani provozní souvislosti, pokud je někdo vědomě nezadal a neověřil. „Funguje to“ a „rozumíme tomu“ nejsou totéž. Mezera mezi nimi se přitom při použití AI nezvětšuje pomalu — otevírá se rychlostí, jakou kód přibývá.

2. Co se ztrácí, když zmizí mentální model

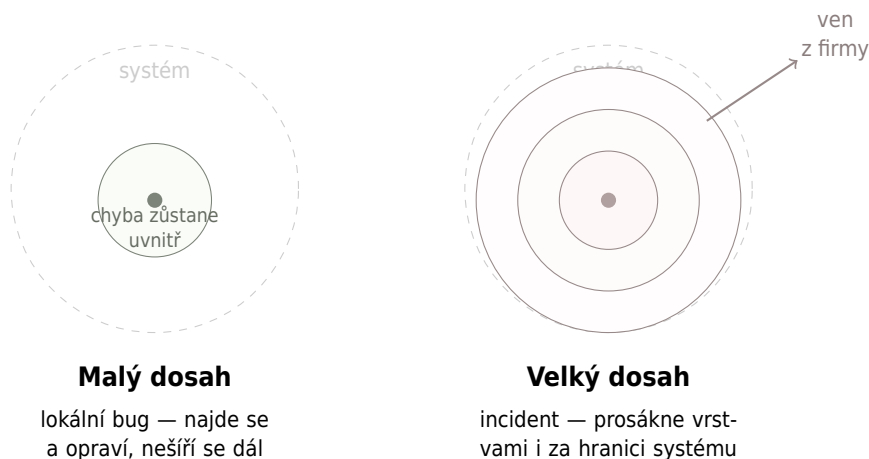
Vstupní bariéra klesla natolik, že lze sestavit funkční aplikaci, aniž by kdokoli prošel fází hlubokého pochopení. Kód existuje, běží — ale mentální model, který by někdo unesl v hlavě, nevznikl.

Důsledky se neprojeví hned. Přijdou později: každá netriviální změna se stává hazardem, protože není jasné, co co ovlivňuje. Ladění se mění z analýzy na metodu pokus-omyl. A když původní autor odejde, kód nemá kdo převzít — ne proto, že by byl složitý, ale proto, že nikdy neexistovalo porozumění, které by se dalo předat.

Tohle není problém AI. Je to starý problém, který AI jen zrychluje a zlevňuje natolik, že se objevuje častěji a dříve.

3. Ne každá chyba je stejná

Intuice, že „chyba je chyba“, je v provozu k ničemu. Chyba ve formátování data je lokální — projeví se, najde se, opraví. Chyba v tom, jak se zachází s daty nebo přístupy, se nemusí projevit vůbec — dokud se neprojeví jako incident, měsíce po nasazení.



Dvě chyby mohou být stejně „velké“ v kódu a přitom mít úplně jiný dosah: jedna zůstane v místě vzniku, druhá se šíří vrstvami systému — a v nejhorším případě až ven, k uživatelům nebo do dat, kde už nejde o bug, ale o incident. Velikost dopadu je ale jen první ze tří otázek.

Proto má smysl ptát se ne „kdo nebo co kód napsal“, ale: jak velkou škodu chyba způsobí, jak rychle ji poznáme a jak snadno ji vrátíme.

- **Ověřitelnost — jak rychle to poznáme.** Některá chyba spadne v testu hned, jiná tiše čeká. Špatná ověřitelnost je zákeřná právě tím, že velký dosah a pozdní odhalení se násobí: než se chyba projeví, je už všude.
- **Reverzibilita — jak snadno to vrátíme.** Vratná migrace nebo přepínač funkce snižuje riziko i u velkého dosahu — omyl prostě vezmeme zpět. Nevratná operace (smazaná data bez zálohy) je nebezpečná i při malém dosahu, protože druhý pokus už není.

Tyto tři otázky — dosah škody, ověřitelnost, reverzibilita — platily dávno před AI. AI je nemění. Jen zvyšuje sázku tím, že umožňuje vyrobit rizikový kód rychleji než kdy dřív.

4. Když je v sázce víc než kód

Některé oblasti nesou víc než jen riziko chyby. Tam, kde se pracuje s osobními údaji nebo s know-how, má nedbalost i právní a obchodní rozměr — regulace jako NIS2 nebo GDPR nejsou jen administrativa, ale popis toho, co se stane, když dosah škody překročí hranici firmy. A nástroj, který odesílá kód či data ke zpracování třetí straně, je vždy odchozím kanálem, ne jen pomocníkem.

Detaily těchto témat patří do samotných pravidel, ne sem. Zde stačí říct, proč na nich záleží: protože tady přestává jít o eleganci kódu a začíná jít o odpovědnost.

5. K čemu rámec míří

Cílem není AI zakázat ani omezit. Není to ani oslava nástroje. Je to snaha držet jednoduchou rovnováhu: rychlost generování vyvážená odpovídající mírou porozumění, ověření a kontroly — nejvíc tam, kde selhání bolí nejvíc.

Zkušený člověk s AI udělá víc práce za méně času, ale stále rozumí každému řádku, který pustí do produkce. To je laťka, ke které pravidla míří. Vše ostatní jsou jen způsoby, jak ji udržet.